Design Patterns

Lukas Holik

lukasholik@cs.aau.dk

(based on slides of Gabriela Montoya, thanks)

A good code is

- Readable
- Modular
- Reusable
- Testable
- Scalable
- Uses encapsulation and abstraction
- Consistent
- Extensible
- Ready for errors
- Efficient
- Secure
- Documented
- ....

What are design patterns

- Design patterns come from **existing solutions to problems.**
- Guidelines and design patterns can be used to **create a good solution.**
- But we have to decide when/how to use each pattern and whether the advantages are enough to overcome possible disadvantages (e.g., increased complexity).

Suggest changes to improve the class Employee

```
class Employee {
    public String firstName, lastName, empId;
    public double experienceInYears;
    public Employee(String firstName, String lastName, double experience) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.experienceInYears = experience;
    }
    public String checkSeniority(double experienceInYears){
       return  experienceInYears > 5 ?"senior":"junior";
    }
}
```

a. Change the access level of the instance fields
b. Change the parameters of the method checkSeniority
c. The field experienceInYears should be static
d. The method checkSeniority should be static

SOLID principles

- **Single Responsibility Principle**
  - A class should have **only one reason to change**, avoid having methods that do different things.
- **Open/Closed Principle**
  - Open for extension, closed to modification. It should be **possible to expand** the set of operations or fields, but the interface must **remain stable** for others to use.
- **Liskov Substitution Principle**
  - It should be possible to **substitute a parent** type with subtype instance.
- **Interface Segregation Principle**
  - Keep in the (super)type **only the necessary methods.**
- **Dependency Inversion Principle**
  - Types should only **depend on more abstract** types.

Marked in blue a few changes from the version in the book.

Example 1

```
class Employee {
    private String firstName, lastName, empId;
    private double experienceInYears;

    public Employee(String firstName, String lastName, double experience) { ... }

    public String toString(){ ... }

    public String checkSeniority(){ ... }

    public String generateEmpId(){ ... }
}
```

An employee that has at least 5 years of experience is senior.

The employee id is composed of the initial letter of the first name and a random integer.

It does not follow:
- Single Responsibility Principle
- Open/Closed Principle.

```
class Employee {
    private String firstName, lastName;
    private double experienceInYears;

    public Employee(String firstName, String lastName, double experience) { ... }

    public String toString(){ ... }
    ...
}
```

```
class SeniorityChecker {

    public String checkSeniority(double experienceInYears){ ... }
}
```

```
class EmployeeIdGenerator {

    public String generateEmpId(String empFirstName){ ... }
}
```

Example 2

```
class Student {
    private String name, regNumber, department;
    private double score;
    public Student(String name, String regNumber, double score, String dept) { ... }
    public String toString() { ... }
    public boolean evaluateDistinction() { ... }
}
```

A student gets a distinction if the score is at least 80 points and their department is within the science stream or at least 70 and their department is within the art stream

It does not follow:
- Single Responsibility Principle
- Open/Closed Principle

```
class Student {
    private String name, regNumber, department;
    private double score;
    public Student(String name, String regNumber, double score, String dept) { ... }
    public String toString() { ... }
    ...
}
```

```
class DistinctionDecider {
    private List<String> science = Arrays.asList("Comp.Sc.", "Physics");
    private List<String> arts = Arrays.asList("History", "English");

    public boolean evaluateDistinction(Student student) {
        boolean distinction = false;
        if (science.contains(student.getDepartment())) {
            distinction = student.getScore() > 80;
        } else if (arts.contains(student.getDepartment())) {
            distinction = student.getScore() > 70;
        }
        return distinction;
    }
}
```

It does not follow:
Open/Closed Principle

New requirement: add an additional stream, e.g., commerce

```
abstract class Student {
    private String name, regNumber, department;
    private double score;
    protected Student(String name, String regNumber, double score, String dept) { ... }
    public String toString() { ... }
    ...
}
```

Rely on abstraction

```
class ArtsStudent extends Student {

    public ArtsStudent(String name, String regNumber, double score, String dept) {
        super(name, regNumber, score, dept);
    }
}
```

```
class ScienceStudent extends Student {

    public ScienceStudent(String name, String regNumber, double score, String dept) {
        super(name, regNumber, score, dept);
    }
}
```

```
interface DistinctionDecider {
    boolean evaluateDistinction(Student student);
}
```

```
class ScienceDistinctionDecider implements DistinctionDecider {

    public boolean evaluateDistinction(Student student) {
        return student.getScore() > 80;
    }
}
```

```
class ArtsDistinctionDecider implements DistinctionDecider {

    public boolean evaluateDistinction(Student student) {
        return student.getScore() > 70;
    }
}
```

Example 3

```
interface Payment {
          void previousPaymentInfo();
          void newPayment();
}
```

```
class RegisteredUserPayment implements Payment {
    private String name; ...
    public RegisteredUserPayment(String userName) { ... }
    public void previousPaymentInfo() { ... }
    public void newPayment() { ... } ...
}
```

```
public class PaymentHelper {
    private List<Payment> payments = new ArrayList<Payment>();
    public void addPayment(Payment payment) { ... }
    public void showPreviousPayments() { ... }
    public void processNewPayments()  { ... }
}
```

New requirement: guest users must be supported

```
interface Payment {
          void previousPaymentInfo();
          void newPayment();
}
```

```
class GuestUserPayment implements Payment {
    private String name;
    public GuestUserPayment() {
        this.name = "guest";
    }
    public void previousPaymentInfo(){
      throw new UnsupportedOperationException();
    }
    public void newPayment() { ... }
}
```

It does not follow:
- Liskov Substitution Principle
- Interface Segregation Principle

```
    public void showPreviousPayments() {
        for (Payment payment: payments) {
          payment.previousPaymentInfo();
          System.out.println("------");
        }
    }
```

Does the following client code end normally?

```
PaymentHelper helper = new PaymentHelper();
GuestUserPayment p = new GuestUserPayment();
helper.addPayment(p);
helper.showPreviousPayments();
```

a. No, an error is thrown
b. No, an exception is thrown
c. Yes

```
interface NewPayment {
          void newPayment();
}
```

```
interface PreviousPayment {
          void previousPaymentInfo();
}
```

```
class RegisteredUserPayment implements NewPayment, PreviousPayment {
    private String name;
    public RegisteredUserPayment(String userName) { ... }
    public void previousPaymentInfo() { ... }
    public void newPayment() { ... }
}
```

```
class GuestUserPayment implements NewPayment {
    private String name;
    public GuestUserPayment() { ... }
    public void newPayment() { ... }
}
```

```
public class PaymentHelper {
    private List<PreviousPayment> previousPayments = new ArrayList<PreviousPayment>();
    private List<NewPayment> newPayments = new ArrayList<NewPayment>();
    public void addPreviousPayment(PreviousPayment previousPayment) { ... }
    public void addNewPayment(NewPayment newPayment) { ... }
    public void showPreviousPayments() { ... }
    public void processNewPayments()  { ... }
}
```

Select the principles followed by these types

```
interface Printer {
        void printDocument();
        void sendFax();
}
```

```
class AdvancedPrinter implements Printer {
        public void printDocument() { ... }
        public void sendFax() { ... }
}
```

a. Single Responsibility Principle
b. Open/Closed Principle
c. Liskov Substitution Principle
d. Interface Segregation Principle
e. Dependency Inversion Principle

New requirement: basic printers without fax capabilities

```
interface Printer {
        void printDocument();
        void sendFax();
}
```

```
class AdvancedPrinter implements Printer {
        public void printDocument() { ... }
        public void sendFax() { ... }
}
```

a. Single Responsibility Principle
b. Open/Closed Principle
c. Liskov Substitution Principle
d. Interface Segregation Principle
e. Dependency Inversion Principle

Refactored printers

```
interface Printer {
        void printDocument();
}
```

```
interface FaxDevice {
        void sendFax();
}
```

```
class BasicPrinter implements Printer {
        public void printDocument() { ... }
}
```

```
class AdvancedPrinter implements Printer, FaxDevice {
        public void printDocument() { ... }
        public void sendFax() { ... }
}
```

Example 4

More concrete or low-level classes changes should not affect more abstract or high-level classes.

```
class UserInterface {
        private OracleDatabase oracleDatabase;

        public UserInterface() {
                this.oracleDatabase = new OracleDatabase();
        }
        public void saveEmployeeId(String empId) {
                oracleDatabase.saveEmpIdInDatabase(empId);
        }
}
```

It does not follow:
- Open/Closed Principle
- Dependency Inversion Principle

```
class OracleDatabase {
        public void saveEmpIdInDatabase(String empId) { ... }
}
```

```
interface Database {
        void saveEmpIdInDatabase(String empId);
}
```

```
class OracleDatabase implements Database {
        public void saveEmpIdInDatabase(String empId) { ... }
}
```

```
class UserInterface {
        private Database database;

        public UserInterface(Database database) {
                this.database = database;
        }
        public void saveEmployeeId(String empId) {
                database.saveEmpIdInDatabase(empId);
        }
}
```

Which principles are followed by the class User?

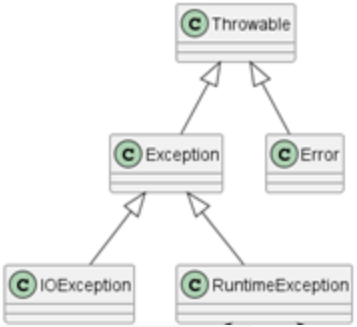
```
import java.time.LocalDate;

class User {
    private String name;
    private LocalDate birthdate;

    public User(String n) {
        this.name = n;
    }
    public String getName() {
        return this.name;
    }
    public void setBirthDate(int year, int month, int day) {
        birthdate = LocalDate.of(year, month, day);
    }
    public LocalDate getBirthDate() {
        return birthdate;
    }
    public boolean isMinor() {
        LocalDate t0 = LocalDate.now().minusYears(18);
        return birthdate.compareTo(t0) >= 0;
    }
}
```

a. Single Responsibility Principle
b. Open/Closed Principle
c. Liskov Substitution Principle
d. Interface Segregation Principle
e. Dependency Inversion Principle

Does the hierarchy follow the Interface Segregation Principle?



Throwable's methods

All Methods | Instance Methods | Concrete Methods

Modifier and Type	Method	Description
final void	addSuppressed(Throwable exception)	Appends the specified exception to the exceptions that were suppressed in order to deliver this exception.
Throwable	fillInStackTrace()	Fills in the execution stack trace.
Throwable	getCause()	Returns the cause of this throwable or null if the cause is nonexistent or unknown.
String	getLocalizedMessage()	Creates a localized description of this throwable.
String	getMessage()	Returns the detail message string of this throwable.
StackTraceElement[]	getStackTrace()	Provides programmatic access to the stack trace information printed by printStackTrace().
final Throwable[]	getSuppressed()	Returns an array containing all of the exceptions that were suppressed, typically by the try-with-resources statement, in order to deliver this exception.
Throwable	initCause(Throwable cause)	Initializes the *cause* of this throwable to the specified value.
void	printStackTrace()	Prints this throwable and its backtrace to the standard error stream.
void	printStackTrace(PrintStream s)	Prints this throwable and its backtrace to the specified print stream.
void	printStackTrace(PrintWriter s)	Prints this throwable and its backtrace to the specified print writer.
void	setStackTrace(StackTraceElement[] stackTrace)	Sets the stack trace elements that will be returned by getStackTrace() and printed by printStackTrace() and related methods.
String	toString()	Returns a short description of this throwable.

Simple Factory Pattern

- (An object with) a method that creates new objects.
- It abstracts the process of object creation (code most likely to change).

```
public static Point2D create(int x, int y) {
    Point2D p = new Point2D();
    p.setX(x);
    p.setY(y);
    return p;
}
```

Example 5


```
interface Animal {
    void displayBehavior();
}
```

```
class Dog implements Animal {
    public void displayBehavior() { ... }
}
```

```
class Tiger implements Animal {
    public void displayBehavior() { ... }
}
```

```
public enum Type {DOG, TIGER};
```

```
class AnimalFactory {
  public Animal createAnimal(Type animalType) {
    Animal animal = null;
    if (animalType.equals(Type.DOG)) {
      animal = new Dog();
    } else if (animalType.equals(Type.TIGER)) {
      animal = new Tiger();
    }
    return animal;
  }
}
```

```
AnimalFactory factory = new AnimalFactory();
Animal animal = factory.createAnimal(Type.DOG);
animal.displayBehavior();
```

New requirement: a new animal must be supported

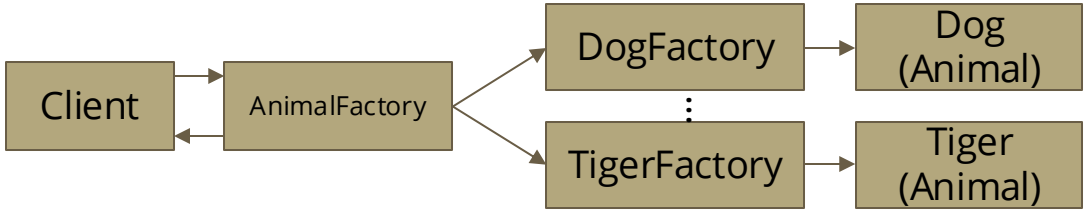
```
public enum Type {DOG, TIGER, CAT};
```

```
class Cat implements Animal {
    void displayBehavior() { ... }
}
```

```
class AnimalFactory {
  public Animal createAnimal(Type animalType) {
    Animal animal = null;
    if (animalType.equals(Type.DOG)) {
      animal = new Dog();
    } else if (animalType.equals(Type.TIGER)) {
      animal = new Tiger();
    }  else if (animalType.equals(Type.CAT)) {
      animal = new Cat();
    }
    return animal;
  }
}
```

It does not follow the Open/Closed Principle

Factory Method Pattern



A type has methods to create objects, but the creation is deferred to its subtypes

```
abstract class AnimalFactory {
  public abstract Animal createAnimal();
}
```

```
class DogFactory extends AnimalFactory {
  public Animal createAnimal() {
      return new Dog();
  }
}
```

```
class CatFactory extends AnimalFactory {
  public Animal createAnimal() {
      return new Cat();
  }
}
```

```
class TigerFactory extends AnimalFactory {
  public Animal createAnimal() {
      return new Tiger();
  }
}
```

```
AnimalFactory factory = new TigerFactory();
Animal animal = factory.createAnimal();
animal.displayBehavior();
```

New requirement: age and name must be supported

```
abstract class Animal {
    private String name;
    private int age;
    public Animal(String name, int age){...}
    void displayBehavior();
}
```

```
class Dog implements Animal {
    public Dog(String name, int age){...}
    void displayBehavior() { ... }
}
```

```
class Tiger implements Animal {
    public Tiger(String name, int age){...}
    void displayBehavior() { ... }
}
```

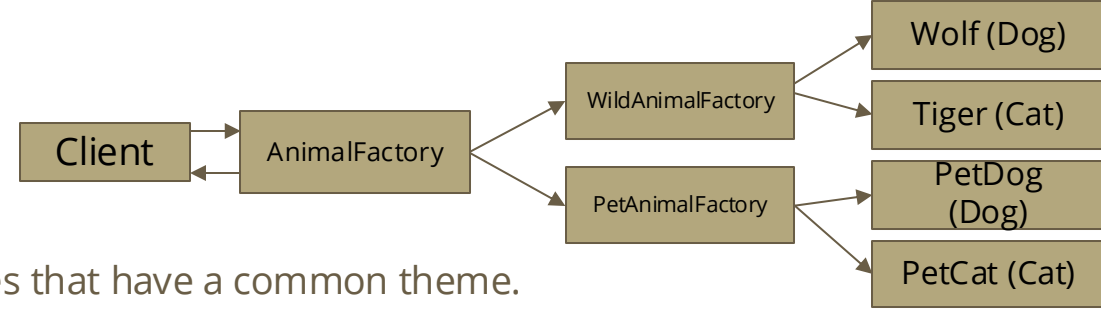
```
abstract class AnimalFactory {
  public abstract Animal createAnimal(
                        String name, int age);
}
```

```
class TigerFactory extends AnimalFactory {
  public Animal createAnimal(String name,
                             int age) {
      return new Tiger(name, age);
  }
}
```

```
class DogFactory extends AnimalFactory {...}
```

```
AnimalFactory factory = new TigerFactory();
Animal animal = factory.createAnimal("Tig", 2);
animal.displayBehavior();
```

Abstract Factory Pattern



Encapsulates a group of individual factories that have a common theme.

Giacomo Ritucci. UML class diagram of the AbstractFactory pattern, https://commons.wikimedia.org/w/index.php?curid=741978

```
interface Dog {
    void displayBehavior();
}
```

```
interface AnimalFactory {
  Cat createCat();
  Dog createDog();
}
```

```
interface Cat {
    void displayBehavior();
}
```

```
class PetDog implements Dog {
    void displayBehavior() { ... }
}
```

```
class PetCat implements Cat {
    void displayBehavior() { ... }
}
```

```
class Wolf implements Dog {
    void displayBehavior() { ... }
}
```

```
class Tiger implements Cat {
    void displayBehavior() { ... }
}
```

```
public class WildAnimalFactory implements AnimalFactory {
  public Cat createCat() { ... }
  public Dog createDog() { ... }
}
```

```
public class PetAnimalFactory implements AnimalFactory {
  public Cat createCat() { ... }
  public Dog createDog() { ... }
}
```

```
AnimalFactory factory
    = new WildAnimalFactory();
Animal animal
    = factory.createCat();
animal.displayBehavior();
```

Consider using the * Factory * pattern for these classes

```
abstract class Student {
    private String name, regNumber, department;
    private double score;
    protected Student(String name, String regNumber, double score, String dept) { ... }
    public String toString() { ... }
    ...
}
```

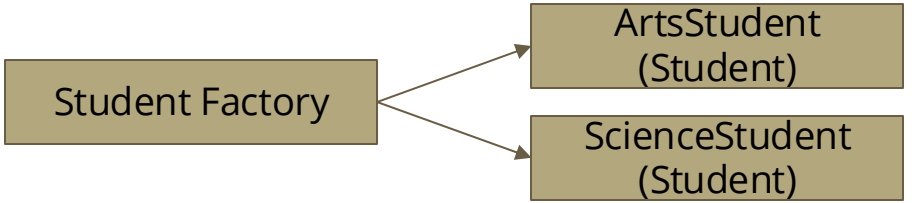
```
class ArtsStudent extends Student {

    public ArtsStudent(String name, String regNumber, double score, String dept) {
        super(name, regNumber, score, dept);
    }
}
```

```
class ScienceStudent extends Student {

    public ScienceStudent(String name, String regNumber, double score, String dept) {
        super(name, regNumber, score, dept);
    }
}
```

```
abstract class Student {
    private String name, regNumber, department;
    private double score;
    protected Student(String name, String regNumber,
              double score, String dept) { ... }
    public String toString() { ... }
    ...
}
```




```
class ArtsStudent extends Student {

    public ArtsStudent(String name, String regNumber,
                       double score, String dept) {
        super(name, regNumber, score, dept);
    }
}
```

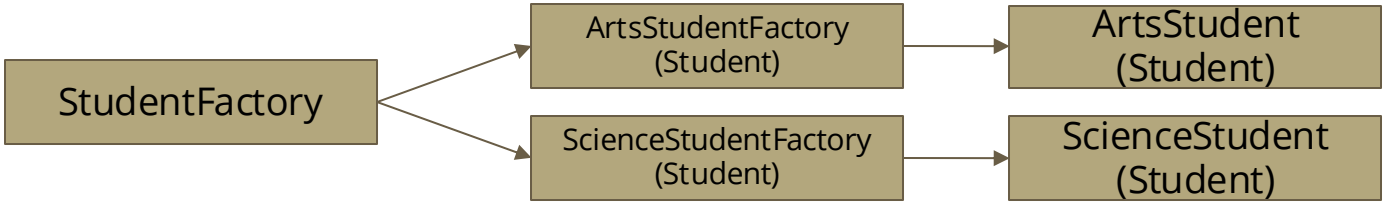
```
class ScienceStudent extends Student {

    public ScienceStudent(String name, String regNumber,
                          double score, String dept) {
        super(name, regNumber, score, dept);
    }
}
```

```
class StudentFactory {
  public Student createStudent(Type studentType,
                        String name, String regNumber,
                        double score, String dept) {
    Student student = null;
    if (studentType.equals(Type.ARTS)) {
      student = new ArtsStudent(name, regNumber, score,
                                dept);
    } else if (studentType.equals(Type.SCIENCE)) {
      student = new ScienceStudent(name, regNumber, score,
                                   dept);
    }
    return student;
  }
}
```

```
public enum Type {ARTS, SCIENCE};
```

```
...
StudentFactory factory = new StudentFactory();
Student s1 = factory.createStudent(Type.ARTS, n, rn, s, d);
```




```
abstract class StudentFactory {
  public Student createStudent(String name,
       String regNumber, double score, String dept);
```

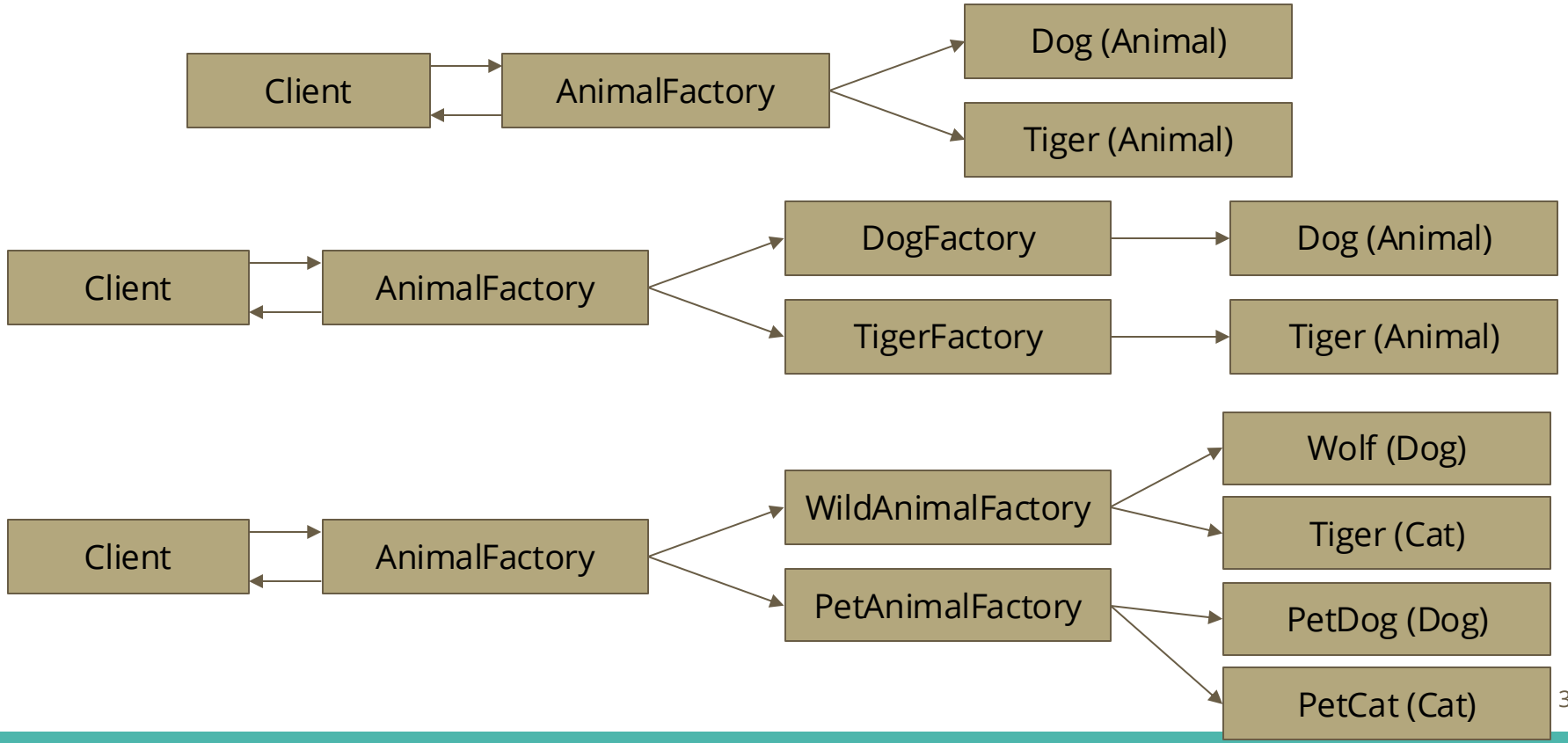
```
...
StudentFactory factory = new
ArtsStudentFactory();
Student s1 = factory.createStudent(n, rn, s, d);
```

```
class ArtsStudentFactory extends StudentFactory {
  public Student createStudent(String name,
       String regNumber, double score, String dept) {
    return new ArtsStudent(name, regNumber, score,
dept);
  }
}
```

```
class ScienceStudentFactory extends StudentFactory {
  public Student createStudent(String name,
       String regNumber, double score, String dept) {
    return new ScienceStudent(name, regNumber, score, dept);
  }
}
```

Promote loose coupling by reducing dependencies on concrete classes

Simple Factory, Factory Method, Abstract Factory



References

- Sarcar, Vaskaran. Java Design Patterns: A Hands-On Experience with Real-World Examples. 3rd edition. Apress, 2022. Chapters 1-4.